

СОФИЙСКА ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО ЕЛЕКТРОНИКА

„Джон Атанасов“

София, ул. „Райко Алексиев“ №48, <https://spge-bg.com/>

ДЪРЖАВЕН ИЗПИТ ЗА ПРИДОБИВАНЕ НА ТРЕТА СТЕПЕН НА
ПРОФЕСИОНАЛНА КВАЛИФИКАЦИЯ – ЧАСТ ПО ТЕОРИЯ НА ПРОФЕСИЯТА

ДИПЛОМЕН ПРОЕКТ

Тема: „Уеб базирано приложение за наемане на автомобили“

Практика: „Разработване на уеб базирано приложение за наемане на автомобили“

Дипломант: Алекс Аксениев Пашов, 12а клас

Професия: 481020 „Системен програмист“

Специалност: 4810201 „Системно програмиране“

Ръководител на дипломен проект: инж. Веселина Маринова

Дипломант:.....

/подпис/

Ръководител на дипломен проект:.....

/подпис/

София, 2026 г.



СЪДЪРЖАНИЕ

1. Въведение.....	3
1.1. Задание за разработка	3
1.2. Анализ на заданието	4
1.3. Поставени цели	4
1.4. Очаквани резултати	5
2. Основна част.....	6
2.1. Избор на технологии.....	6
2.1.1. Избор на технология за сървърната част	6
2.1.2. Избор на технология за клиентска част	7
2.1.3. Избор на технология за базата данни	8
2.2. Софтуерен дизайн.....	8
2.2.1. Етапи на разработка	8
2.2.2. Софтуерна архитектура	9
2.2.3. Описание и взаимодействие с приложението	9
2.2.5. Дизайн на сървъра и CRUD заявки	18
2.2.6. Графичен дизайн и структура на приложението.....	32
3. Заключение.....	33
3.1. Постигнати цели.....	33
3.2. Бъдещо развитие	34
4. Използвана литература	35
5. Приложения	37
Приложение 5.1. Структура на проекта.....	37
Приложение 5.2. Тестови сценарии	37
Приложение 5.3. Тестови акаунти	39



1. Въведение

Настоящият дипломен проект представя разработката на пълнофункционална уеб система за управление на луксозен автомобилен ризорт, наречена "Luxe Auto Resort". Системата е предназначена да автоматизира и оптимизира процесите по наемане на автомобили, управление на клиентски заявки и администриране на автопарк. Проектът обхваща цялостния жизнен цикъл на разработката – от анализ на изискванията и проектиране до реализация и тестване на функционален уеб продукт.

Съвременният пазар на луксозни автомобили изисква дигитализация на всички бизнес процеси. Традиционните методи за управление на автопарк и обработка на заявки за наем вече не отговарят на очакванията на взискателните клиенти, нито на нуждите на бизнеса за ефективност и прозрачност. Разработеното приложение предлага модерно решение, което обединява в единна платформа функционалностите, необходими на клиентите, и инструментите за управление, необходими на администраторите.

1.1. Задание за разработка

В съвременния свят наемането на луксозни автомобили се е превърнало в предпочитана алтернатива на покупката, особено за бизнес пътувания, специални поводи и туристически цели. Въпреки нарастващото търсене, голяма част от компаниите, опериращи в този сегмент, все още разчитат на остарели методи за управление – телефонни резервации, хартиени формуляри и неинтегрирани системи. Тези подходи водят до грешки, неефективност и лошо клиентско изживяване.

Задачата на настоящия дипломен проект е да се разработи уеб базирана платформа, която да реши именно тези проблеми. "Luxe Auto Resort" е система, която предоставя на клиентите интуитивен начин за преглед и резервация на луксозни автомобили, а на администраторите – мощни инструменти за управление на автопарка, обработка на заявки и следене на приходите.

Основните проблеми, които системата решава, са следните: първо, липсата на централизирана платформа за управление на автопарка и клиентските заявки; второ, невъзможността на клиентите да преглеждат наличните автомобили и да резервират онлайн по всяко време; трето, трудността при изчисляване на цени с различни отстъпки и шаблони за наем; и четвърто, липсата на система за модерация на коментари и обратна връзка от клиентите.



Системата е проектирана да обслужва два основни типа потребители: клиенти, които търсят и резервират автомобили, и администратори, които управляват цялата платформа. Разграничението между двете роли е реализирано чрез система за автентикация и авторизация, базирана на JWT (JSON Web Tokens).

1.2. Анализ на заданието

За да бъде разработено ефективно решение, е необходимо задълбочено разбиране на нуждите на различните категории потребители на системата. Анализът на изискванията е проведен на база наблюдение на типичните workflows при управление на автомобилен ризорт и консултации с потенциални потребители.

Клиентите на платформата имат следните основни нужди: възможност за преглед на пълен каталог с налични автомобили с детайлна информация за всеки; функционалност за филтриране по марка, тип, цена и наличност; лесен процес на регистрация и влизане в системата; удобна форма за създаване на заявка за наем с избор на дати и автомобили; изчисляване на цената в реално време с прилагане на отстъпки от шаблони; преглед на историята на своите заявки и техния статус; и възможност за оставяне на коментари и оценки.

Администраторите имат значително по-разширени нужди от функционалност: пълен контрол върху каталога с автомобили чрез CRUD операции (добавяне, редактиране, изтриване); управление на шаблони за наем с различна продължителност и отстъпки; преглед и обработка на всички клиентски заявки с функционалност за смяна на статус; модерация на коментари за платформата и за шаблоните; преглед на контактни съобщения от клиенти; достъп до статистически данни в dashboard; и управление на разходите по клиенти.

Системата трябва да поддържа и редица нефункционални изисквания, свързани с производителността, сигурността и достъпността. По отношение на сигурността, всички пароли трябва да се съхраняват хеширани с bcrypt алгоритъм, а достъпът до защитените ресурси – да се контролира чрез JWT токени. Що се отнася до достъпността, системата трябва да работи на всички съвременни устройства – от персонални компютри до мобилни телефони.

1.3. Поставени цели

Целта на настоящия дипломен проект е да се разработи модерен, функционален и сигурен уеб продукт, който да автоматизира процесите на управление на луксозен автомобилен ризорт. Проектът трябва да демонстрира умения за пълноциклова разработка на



уеб приложение, включваща проектиране на база данни, разработка на REST API, изграждане на потребителски интерфейс и осигуряване на сигурност.

Конкретните технически цели на проекта са следните: изграждане на RESTful API сървър с Node.js и Express, реализиращ всички необходими endpoints за комуникация между клиентската и сървърната части; проектиране и реализация на релационна база данни с MySQL, съдържаща всички необходими таблици и връзки между тях; разработка на потребителски интерфейс с HTML, CSS и JavaScript, осигуряващ интуитивно и приятно потребителско изживяване; имплементация на система за автентикация и авторизация с JWT и bcrypt; и реализация на пълен набор CRUD операции за всички основни обекти в системата.

1.4. Очаквани резултати

В резултат на успешното изпълнение на проекта се очаква да бъде реализиран пълнофункционален уеб продукт, готов за реална употреба. Системата трябва да демонстрира стабилна работа при едновременно взаимодействие от множество потребители и да осигурява надеждно съхранение и обработка на данните.

От техническа гледна точка, очакваните резултати включват: работещ сървър с Node.js и Express, обработващ HTTP заявки и осигуряващ сигурен достъп до данните; добре нормализирана MySQL база данни с правилно зададени индекси за оптимизирано извличане на данни; responsive потребителски интерфейс, работещ на всички популярни браузъри и устройства; система за автентикация и авторизация, осигуряваща защита на чувствителните данни; и пълен набор от REST API endpoints, следващи добрите практики за дизайн на API.



2. Основна част

2.1. Избор на технологии

При изграждането на съвременно уеб приложение изборът на подходящи технологии е от ключово значение за неговата ефективност, сигурност и мащабируемост. За разработката на "Luxe Auto Resort" са разгледани и сравнени водещите технологии за сървърна и клиентска разработка, за да се вземат обосновани решения, съобразени с конкретните изисквания на проекта.

2.1.1. Избор на технология за сървърната част

Сървърната логика на уеб приложението е отговорна за обработката на HTTP заявки, управлението на бизнес логиката, комуникацията с базата данни и осигуряването на сигурността. При избора на технология за тази цел са разгледани три основни опции.

PHP (PHP: Hypertext Preprocessor) е скриптов език за програмиране, създаден специално за разработка на динамични уеб страници. Той е с отворен код, широко разпространен и с огромна общност от разработчици. Основните предимства на PHP включват лесна интеграция с уеб сървъри (Apache, Nginx), вградена поддръжка за работа с различни бази данни (MySQL, PostgreSQL) и относително нисък праг за навлизане. Недостатъците на PHP са свързани с уязвимости при неправилна употреба (SQL инжекции, XSS атаки), исторически несъответствия в синтаксиса, и по-слабата производителност в сравнение с компилирани решения.

Python с Django или Flask е друга популярна опция за уеб разработка. Django предоставя „всичко включено“ подход с вграден ORM, административен панел и сигурностни механизми. Flask е по-лек микрофреймуърк с повече свобода за архитектурни решения. Недостатъците включват по-бавна производителност при I/O интензивни операции в сравнение с Node.js и по-висок overhead при малки приложения.

Node.js е среда за изпълнение на JavaScript от страна на сървъра, изградена върху Google V8 JavaScript двигателя. Тя позволява използването на JavaScript едновременно за клиентска и сървърна разработка. Node.js използва асинхронен, събитийно-ориентиран модел, което го прави особено подходящ за I/O интензивни операции. NPM (Node Package Manager) предоставя достъп до огромна екосистема от модули. Express.js е минималистична рамка за



Node.js, предоставяща лесно маршрутизиране, middleware поддръжка и управление на HTTP заявки.

Таблица 2.1.1. Сравнение на технологии за сървърна разработка

Критерий	PHP	Python/Django	Node.js + Express
Производителност	Средна	Средна	Висока (асинхронна)
Лесно учене	Лесно	Средно	Средно
Екосистема	Голяма	Голяма	Огромна (NPM)
REST API поддръжка	Добра	Отлична	Отлична
Единен език (JS)	Не	Не	Да
Мащабируемост	Добра	Добра	Отлична

На база на направения анализ за сървърната част на "Luxe Auto Resort" е избрана комбинацията Node.js и Express.js поради единния програмен език, отличната производителност и богатата NPM екосистема.

2.1.2. Избор на технология за клиентска част

Клиентската (frontend) часта е отговорна за визуалното представяне на данните и взаимодействието с потребителя. Разгледани са HTML5, CSS3, Vanilla JavaScript и JavaScript рамките React.js, Vue.js и Angular.

HTML5 въведе семантични елементи (header, nav, section, footer), подобрени форми и мултимедийна поддръжка. CSS3 добави Custom Properties (CSS Variables), Flexbox, Grid, Media Queries, Transitions и Animations. Vanilla JavaScript (ES6+) предлага Arrow functions, Fetch API, Async/Await, Destructuring и LocalStorage.

React.js, Vue.js и Angular предлагат компонентен подход и управление на state, но носят допълнителна сложност и зависимости. За "Luxe Auto Resort" е избрана комбинацията HTML5 + CSS3 + Vanilla JavaScript, тъй като приложението не изисква сложно управление на state, а липсата на зависимости осигурява минимален размер на файловете и максимална съвместимост.



2.1.3. Избор на технология за базата данни

Разгледани са три системи: MySQL, PostgreSQL и MongoDB. Релационните бази данни (MySQL, PostgreSQL) са подходящи за структурирани данни с добре дефинирани връзки, осигуряват ACID транзакции и поддържат JOIN операции. MongoDB е документно-ориентирана нерелационна СУБД, подходяща за данни с динамична структура.

Таблица 2.1.3. Сравнение на СУБД

Критерий	MySQL	PostgreSQL	MongoDB
Тип	Релационна	Релационна	Нерелационна
ACID транзакции	Да (InnoDB)	Да	Частично
JOIN операции	Да	Да	Не (lookup)
Node.js поддръжка	Отлична (mysql2)	Отлична (pg)	Отлична (mongoose)
Схема	Строга	Строга	Гъвкава

За "Luxe Auto Resort" е избрана MySQL с InnoDB engine: данните са добре структурирани с ясни релации между таблиците, mysql2 библиотеката осигурява отлична Node.js интеграция, а InnoDB поддържа ACID транзакции и референтна цялост чрез чужди ключове.

2.2. Софтуерен дизайн

2.2.1. Етапи на разработка

Разработката на "Luxe Auto Resort" следва методологията на жизнения цикъл на разработка на софтуер (SDLC). Процесът преминава последователно през седем етапа: (1) Събиране и анализ на изискванията; (2) Планиране – определяне на технологичния стек и архитектурата; (3) Проектиране – ER диаграма, API спецификация и wireframes; (4) Кодиране – паралелна разработка на backend, база данни и frontend; (5) Тестване – ръчно тестване на всички CRUD операции и responsive дизайн; (6) Внедряване – конфигуриране за локална и LAN среда; и (7) Поддръжка – корекции и документация.



2.2.2. Софтуерна архитектура

"Luxe Auto Resort" следва REST (Representational State Transfer) архитектурен стил с разделена клиент-сървър архитектура. Сървърната архитектура е организирана в слоеве: маршрутизиращ слой (Routes), middleware слой (автентикация, авторизация), бизнес логика (route handlers) и слой за достъп до данни (MySQL заявки). Клиентската архитектура се гради около централния main.js модул с обекти auth, api и ui. Комуникацията е чрез HTTP с JSON формат и унифициран отговор.

2.2.3. Описание и взаимодействие с приложението

"Luxe Auto Resort" поддържа три нива на достъп. Анонимният посетител може да разглежда каталога с автомобили, да филтрира по марка/тип/цена/наличност, да преглежда шаблоните и одобрените коментари, и да изпраща контактни съобщения. Регистрираният клиент получава допълнително: създаване на заявки с множествен избор на автомобили, изчисляване на цена в реално време, преглед на историята на заявките и оставяне на коментари. Потребителският профил предоставя интерфейс за управление на личната информация, позволявайки редакция на име, фамилия, телефонен номер и имейл адрес чрез форма с вградена валидация и индикация за успешно запазване. **Виж снимка номер 5.4.3.**

Страницата „Моите заявки“ визуализира пълната история на резервациите на клиента, представяйки детайли за избраните автомобили, период на наемане, обща изчислена цена с приложени шаблонни отстъпки, както и текущия статус на всяка заявка в жизнения цикъл (pending, approved, rejected, cancelled). **Виж снимка номер 5.4.4.**

Администраторът разполага с пълен административен панел: dashboard с обобщена статистика, CRUD операции за автомобили и шаблони, обработка на заявки, модерация на коментари и управление на контактни съобщения.

Административният панел за управление на автопарка предоставя табличен изглед на всички записи в таблицата cars, с индикация за текуща наличност, бързи контроли за редакция и изтриване на записи, както и функционалност за добавяне на нови автомобили, реализирайки пълния набор CRUD операции. **Виж снимка номер 5.4.5.**

Разделът за обработка на клиентски заявки от администраторска гледна точка визуализира всички резервации с информация за клиента (JOIN с users), избраните автомобили и предоставя контроли за смяна на статуса (PATCH операция) и филтриране по статус. Виж снимка номер 5.4.6.



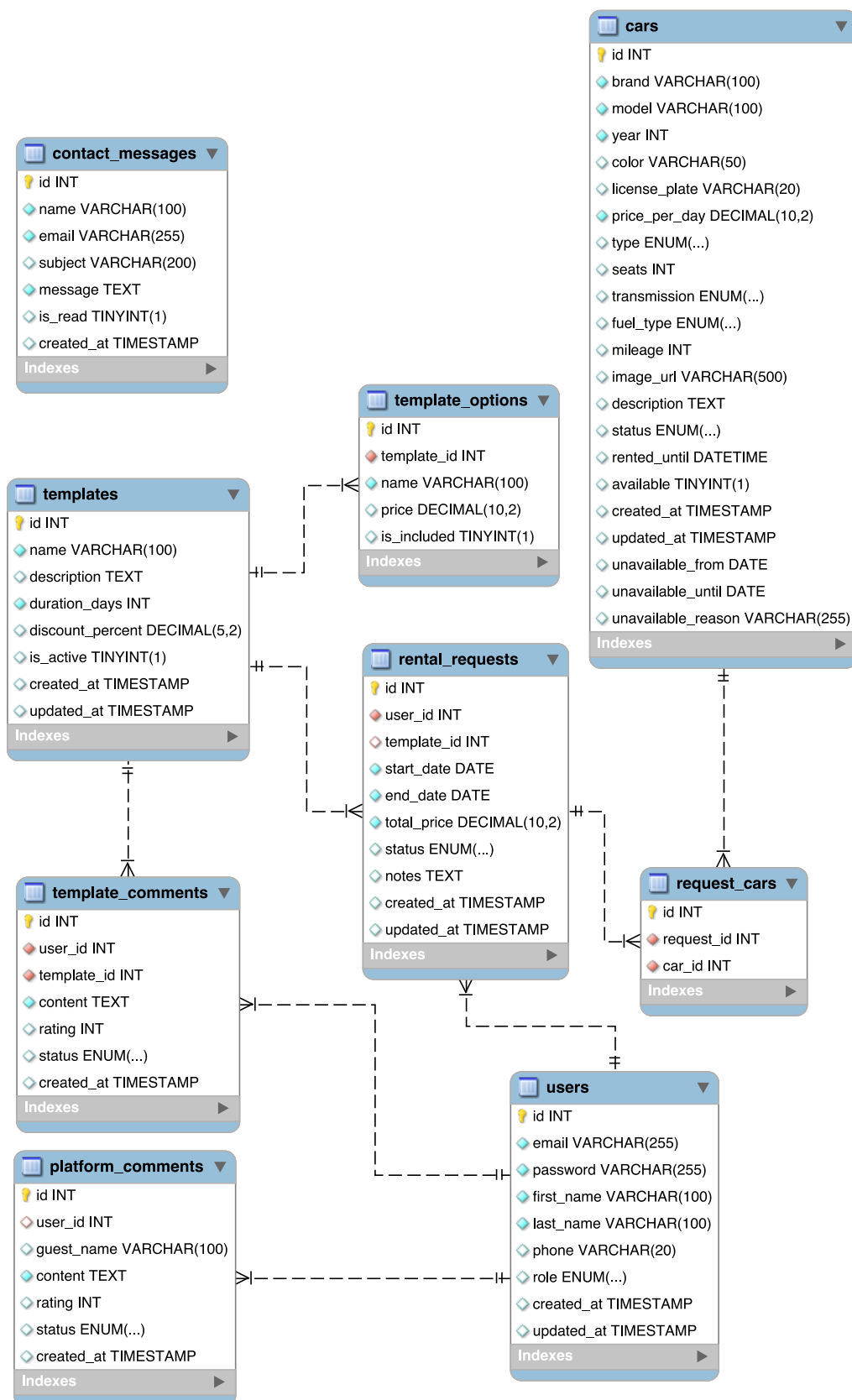
Началната страница на платформата реализира херо секция с интегрирана търсачка за бързо намиране на автомобили по марка, тип и ценови диапазон, навигационно меню с адаптивен дизайн и визуална презентация с тъмна цветова палитра, създаваща усещане за луксозност. *Виж снимка номер 5.4.1.*

Каталогът с автомобили използва CSS Grid за адаптивно разполагане на карти, всяка от които визуализира изображение на автомобила, марка, модел, ключови характеристики (скоростна кутия, гориво, брой места) и цена на ден, с интегрирани филтри и опции за сортиране. *Виж снимка номер 5.4.2.*

2.2.4. Дизайн на базата данни

Базата данни на "Luxe Auto Resort" е проектирана като релационна MySQL база данни с InnoDB storage engine. Съдържа девет таблици, свързани чрез първични и чужди ключове. ER диаграмата на базата е представена по-долу (вж. Фиг. 2.2.4.1).

Дизайнът спазва принципите на Третата нормална форма (3NF): всяко поле зависи само от първичния ключ и не съществува транзитивна зависимост между непървични атрибути. Многогото-към-много връзките се реализират чрез свързващи (junction) таблици. Индексите са поставени стратегически върху полетата, най-често използвани за търсене и JOIN операции, за да се минимизира броят на пълните сканирания на таблиците.



Фиг. 2.2.4.1. ER диаграма на базата данни на "Luxe Auto Resort"



SQL дефиницията на базата данни се намира в /database/schema.sql. Файлът създава базата с utf8mb4 кодиране, осигуряващо поддръжка на кирилица и специални символи:

```
CREATE DATABASE IF NOT EXISTS luxe_auto_resort
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE luxe_auto_resort;
```

Таблица users (Потребители)

Таблицата users е централна за цялата система. Тя съхранява акаунтите на всички регистрирани потребители и дефинира тяхната роля. Структурата е следната:

Таблица 2.2.4.1. Полета на таблица users

Поле	Тип / Описание
id	INT AUTO_INCREMENT PRIMARY KEY – уникален идентификатор на потребителя
email	VARCHAR(255) NOT NULL UNIQUE – имейл адрес, използван за вход; индексирани (idx_email)
password	VARCHAR(255) NOT NULL – bcrypt хеш на паролата (никога не се съхранява открито)
first_name	VARCHAR(100) NOT NULL – собствено име
last_name	VARCHAR(100) NOT NULL – фамилно име
phone	VARCHAR(20) – телефонен номер (незадължително)
role	ENUM('admin','client') DEFAULT 'client' – роля; индексирани (idx_role)
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

SQL дефиниция:

```
CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  email       VARCHAR(255) NOT NULL UNIQUE,
  password    VARCHAR(255) NOT NULL,
  first_name  VARCHAR(100) NOT NULL,
  last_name   VARCHAR(100) NOT NULL,
```



```
phone      VARCHAR(20),
role       ENUM('admin','client') DEFAULT 'client',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
INDEX idx_email (email),
INDEX idx_role (role)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Таблица cars (Автомобили)

Таблицата cars съдържа пълна информация за всеки автомобил в автопарка. Тя е обогатена с полета за управление на наличността: освен булевото поле available, съдържа и полета unavailable_from / unavailable_until за планирани технически престои (напр. профилактика), и rented_until за проследяване на текущи наеми.

Поleta на таблица cars SQL дефиниция (съкратена):

```
CREATE TABLE cars (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  brand       VARCHAR(100) NOT NULL,
  model       VARCHAR(100) NOT NULL,
  year        INT NOT NULL,
  color       VARCHAR(50),
  license_plate VARCHAR(20) UNIQUE,
  price_per_day DECIMAL(10,2) NOT NULL,
  type        ENUM('sedan','suv','coupe','minivan','truck','sport')
              DEFAULT 'sedan',
  seats       INT DEFAULT 5,
  transmission ENUM('manual','automatic') DEFAULT 'automatic',
  fuel_type    ENUM('petrol','diesel','electric','hybrid') DEFAULT
'petrol',
  mileage     INT DEFAULT 0,
  image_url   VARCHAR(500),
  description  TEXT,
  status       ENUM('available','rented','maintenance'),
  rented_until DATETIME,
  available    TINYINT(1) DEFAULT 1,
  unavailable_from DATE,
  unavailable_until DATE,
  unavailable_reason VARCHAR(255),
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```



```
updated_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
INDEX idx_brand      (brand),
INDEX idx_model      (model),
INDEX idx_price      (price_per_day),
INDEX idx_available  (available),
INDEX idx_type       (type)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Таблица templates (Шаблони за наем)

Шаблоните (пакетите) са предварително конфигурирани офери с фиксирана продължителност и отстъпка. Клиентът може да избере шаблон при създаване на заявка и автоматично получава съответната отстъпка.

```
CREATE TABLE templates (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  name            VARCHAR(100) NOT NULL,
  description     TEXT,
  duration_days   INT NOT NULL,
  discount_percent DECIMAL(5,2) DEFAULT 0,
  is_active       TINYINT(1) DEFAULT 1,
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  INDEX idx_active (is_active)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Трите стандартни шаблона, заредени при инициализация на базата (seed.sql):

```
INSERT INTO templates (name, description, duration_days, discount_percent)
VALUES
  ('Уикенд', 'За кратка почивка. Неограничен пробег.', 3, 10.00),
  ('Седмичен', 'За седмична почивка или бизнес пътуване.', 7, 15.00),
  ('Бизнес', 'Проф. пакет с GPS и детско столче.', 30, 25.00);
```

Таблица template_options (Опции за шаблони)

Всеки шаблон може да има произволен брой опции (екстри). Опциите с is_included = TRUE са включени в цената, а тези с is_included = FALSE са допълнителни доплащания.



```
CREATE TABLE template_options (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  template_id INT NOT NULL,  
  name        VARCHAR(100) NOT NULL,  
  price       DECIMAL(10,2),  
  is_included TINYINT(1) DEFAULT 0,  
  FOREIGN KEY (template_id) REFERENCES templates(id) ON DELETE CASCADE,  
  INDEX idx_template (template_id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Таблица rental_requests (Заявки за наем)

Централната бизнес таблица. Всяка заявка принадлежи на точно един потребител и по избор на един шаблон. Общата цена се изчислява при създаването и се записва в базата, за да не зависи от бъдещи промени в ценоразписа.

Таблица 2.2.4.2. Жизнен цикъл на статуса на заявка

Статус	Значение	Следващ статус
pending	Чакаща — изчаква административно одобрение	approved / rejected / cancelled
approved	Одобрена — автомобилите са резервирани	completed / cancelled
rejected	Отказана от администратор	— (краен)
completed	Приключена — наемът е завършен	— (краен)
cancelled	Отменена от клиента или администратора	— (краен)

```
CREATE TABLE rental_requests (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  user_id     INT NOT NULL,  
  template_id INT,  
  start_date  DATE NOT NULL,  
  end_date    DATE NOT NULL,  
  total_price DECIMAL(10,2) NOT NULL,  
  status      ENUM('pending','approved','rejected','completed','cancelled')  
              DEFAULT 'pending',  
  notes       TEXT,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```



```
updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
FOREIGN KEY (user_id)      REFERENCES users(id) ON DELETE CASCADE,
FOREIGN KEY (template_id) REFERENCES templates(id) ON DELETE SET NULL,
INDEX idx_user      (user_id),
INDEX idx_status    (status),
INDEX idx_dates      (start_date, end_date),
INDEX idx_created    (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Таблица request_cars (Автомобили в заявка)

Реализира многото-към-много връзката между rental_requests и cars. Един наем може да включва множество автомобили (напр. основен автомобил + пикап). UNIQUE ограничението предотвратява добавянето на един и същ автомобил два пъти в една заявка.

```
CREATE TABLE request_cars (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  request_id  INT NOT NULL,
  car_id      INT NOT NULL,
  FOREIGN KEY (request_id) REFERENCES rental_requests(id) ON DELETE
CASCADE,
  FOREIGN KEY (car_id)      REFERENCES cars(id)              ON DELETE
CASCADE,
  UNIQUE KEY unique_request_car (request_id, car_id),
  INDEX idx_request (request_id),
  INDEX idx_car      (car_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Таблицы platform_comments и template_comments (Коментари)

Системата поддържа два вида коментари: за платформата като цяло и за конкретни шаблони. И двете таблици имат поле status с три стойности (pending / approved / rejected), което реализира workflow за модерация: новите коментари влизат с pending и стават видими едва след admin одобрение. Незадължителното поле guest_name позволява анонимни коментари в platform_comments.

```
CREATE TABLE platform_comments (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT,
```




```
guest_name VARCHAR(100),
content     TEXT NOT NULL,
rating      INT CHECK (rating >= 1 AND rating <= 5),
status      ENUM('pending','approved','rejected') DEFAULT 'pending',
created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
INDEX idx_user    (user_id),
INDEX idx_status  (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE template_comments (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT NOT NULL,
  template_id INT NOT NULL,
  content     TEXT NOT NULL,
  rating      INT CHECK (rating >= 1 AND rating <= 5),
  status      ENUM('pending','approved','rejected') DEFAULT 'pending',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id)      REFERENCES users(id)      ON DELETE CASCADE,
  FOREIGN KEY (template_id) REFERENCES templates(id) ON DELETE CASCADE,
  INDEX idx_template (template_id),
  INDEX idx_status   (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Таблица `contact_messages` (Контактни съобщения)

Съхранява съобщенията, изпратени чрез контактната форма. Не изисква автентикация — достъпна е за всички посетители. Булевото поле `is_read` позволява администраторите да следят кои съобщения са обработени.

```
CREATE TABLE contact_messages (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(255) NOT NULL,
  subject     VARCHAR(200),
  message     TEXT NOT NULL,
  is_read     TINYINT(1) DEFAULT 0,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_read    (is_read),
  INDEX idx_created (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```



Таблица 2.2.4.3. Обобщение на таблиците и техните връзки

Таблица	Записи	Ключови полета	Връзки
users	Потребители	id, email, role	—
cars	Автомобили	id, brand, price_per_day, available	—
templates	Шаблони	id, name, discount_percent	—
template_options	Екстри за шаблони	template_id, name, is_included	templates (1:N)
rental_requests	Заявки за наем	id, user_id, total_price, status	users (N:1), templates (N:1)
request_cars	Автомобили в заявка	request_id, car_id	rental_requests (N:1), cars (N:1)
platform_comments	Коментари платформа	user_id, rating, status	users (N:1)
template_comments	Коментари шаблони	user_id, template_id, status	users (N:1), templates (N:1)
contact_messages	Контактни съобщения	name, email, is_read	—

2.2.5. Дизайн на сървъра и CRUD заявки

Сървърът е реализиран с Node.js и Express.js. Основният файл server.js конфигурира CORS, JSON body parser, статичните файлове и регистрира всички маршрутни модули:

```
// backend/server.js (извадка)
const app = express();
app.use(cors({ origin: FRONTEND_URL, credentials: true }));
app.use(express.json());
app.use(express.static(path.join(__dirname, '../frontend')));

app.use('/api/auth',      require('./routes/auth'));
app.use('/api/cars',      require('./routes/cars'));
app.use('/api/templates', require('./routes/templates'));
```



```
app.use('/api/requests', require('./routes/requests'));
app.use('/api/costs', require('./routes/costs'));
app.use('/api/contacts', require('./routes/contacts'));
app.use('/api/comments', require('./routes/comments'));

app.use(notFoundHandler); // 404
app.use(errorHandler); // централизирана обработка на грешки
```

Централизирана обработка на грешки (utils/errors.js)

Вместо да пише try/catch в края на всеки route handler, проектът използва asyncHandler — wrapper функция, която прихваща всяко хвърлено изключение и го предава на Express error middleware:

```
// backend/utils/errors.js
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// Стандартизиран формат на отговора при грешка:
const errorHandler = (err, req, res, next) => {
  if (err.isOperational) { // познати грешки
    return res.status(err.statusCode).json(err.toJSON());
  }
  if (err.name === 'TokenExpiredError') {
    return res.status(401).json({
      success: false,
      error: { message: 'Токенът е изтекъл.', code: 'TOKEN_EXPIRED' }
    });
  }
  if (err.code === 'ER_DUP_ENTRY') { // MySQL duplicate key
    return res.status(409).json({
      success: false,
      error: { message: 'Записът вече съществува.', code: 'ALREADY_EXISTS' }
    });
  }
  res.status(500).json({ success: false,
    error: { message: 'Вътрешна грешка на сървъра.', code: 'INTERNAL_ERROR' }
  });
};
```



Middleware за автентикация и авторизация

Файлът `middleware/auth.js` дефинира две функции, прилагани като `middleware` в защитените маршрути:

```
// backend/middleware/auth.js
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET ||
'luxe_auto_resort_secret_key_2024';

const requireAuth = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token)
    return res.status(401).json({ error: 'Трябва да сте влезли в профила си' });
  try {
    req.user = jwt.verify(token, JWT_SECRET); // поставя {id, email, role}
    next();
  } catch (e) {
    return res.status(401).json({ error: 'Невалиден токен' });
  }
};

const requireAdmin = (req, res, next) => {
  if (req.user?.role !== 'admin')
    return res.status(403).json({ error: 'Изисква се администратор' });
  next();
};
```

CRUD за автентикация (`routes/auth.js`)

При регистрацията (CREATE потребител) сървърът валидира входа, хешира паролата с `bcrypt` и записва новия потребител:

```
// POST /api/auth/register
router.post('/register', asyncHandler(async (req, res) => {
  const { email, password, first_name, last_name, phone } = req.body;

  // 1. Валидация
  const errors = validateRegistrationInput(req.body);
  if (errors.length > 0)
    return res.status(400).json({
      success: false,
```



```
      error: { message: 'Моля, поправете грешките.', code:
'VALIDATION_ERROR',
              details: errors }
    });

    // 2. Проверка за дублиран имейл
    const [existing] = await pool.query(
      'SELECT id FROM users WHERE email = ?', [email.toLowerCase().trim()]
    );
    if (existing.length > 0)
      return res.status(409).json({
        success: false,
        error: { message: 'Имейлът е зает.', code: 'EMAIL_ALREADY_EXISTS' }
      });

    // 3. Хеширане и запис
    const hashed = await bcrypt.hash(password, 10); // cost factor 10
    await pool.query(
      'INSERT INTO users (email,password,first_name,last_name,phone,role)',
      + ' VALUES (?, ?, ?, ?, ?, ?)',
      [email.toLowerCase(), hashed, first_name, last_name, phone || null,
'client']
    );
    res.status(201).json({ success: true, message: 'Успешна регистрация' });
  });
});
```

При вход (READ потребител + генериране на JWT), паролата се верифицира с `bcrypt.compare()`. Токенът включва `id`, `email` и `role` — достатъчно за авторизация без допълнителна заявка към базата:

```
// POST /api/auth/login
router.post('/login', asyncHandler(async (req, res) => {
  const { email, password } = req.body;

  const [users] = await pool.query(
    'SELECT * FROM users WHERE email = ?', [email.toLowerCase().trim()]
  );
  if (users.length === 0)
    return res.status(401).json({
      success: false, error: { message: 'Невалиден имейл или парола.',
                                code: 'INVALID_CREDENTIALS' }
    });
});
```



```
const valid = await bcrypt.compare(password, users[0].password);
if (!valid) /* — същата грешка, без да разкриваме кое е грешното */ ;

const token = jwt.sign(
  { id: users[0].id, email: users[0].email, role: users[0].role },
  JWT_SECRET,
  { expiresIn: '24h' }
);
res.json({ success: true, token, user: { ...users[0], password: undefined } });
}));
```

CRUD за автомобили (routes/cars.js)

READ — четене на всички автомобили с динамично филтриране:

Заявката GET /api/cars поддържа осем query параметъра: brand, type, min_price, max_price, available и sort. SQL заявката се изгражда динамично — ако даден параметър липсва, условието не се добавя в WHERE клаузата. Това осигурява един endpoint вместо множество специализирани:

```
// GET /api/cars?brand=BMW&type=suv&min_price=100&sort=price_asc
router.get('/', asyncHandler(async (req, res) => {
  const { brand, type, min_price, max_price, available, sort } = req.query;
  let query = 'SELECT * FROM cars WHERE 1=1';
  const params = [];

  if (brand) { query += ' AND brand LIKE ?';
  params.push(`%${brand}%`); }
  if (type) { query += ' AND type = ?'; params.push(type); }
  if (min_price) { query += ' AND price_per_day >= ?';
  params.push(min_price); }
  if (max_price) { query += ' AND price_per_day <= ?';
  params.push(max_price); }
  if (available === 'true') query += ' AND available = TRUE';

  switch (sort) {
    case 'price_asc': query += ' ORDER BY price_per_day ASC'; break;
    case 'price_desc': query += ' ORDER BY price_per_day DESC'; break;
    case 'year_desc': query += ' ORDER BY year DESC'; break;
    case 'brand': query += ' ORDER BY brand, model'; break;
  }
})
```



```
default:          query += ' ORDER BY id';
}

const [cars] = await pool.query(query, params);
res.json({ success: true, data: cars, count: cars.length });
});
```

CREATE — добавяне на нов автомобил (само за администратори):

```
// POST /api/cars
router.post('/', requireAuth, requireAdmin, asyncHandler(async (req, res)
=> {
  const { brand, model, year, license_plate, price_per_day,
        type, seats, transmission, fuel_type, mileage,
        image_url, description, available } = req.body;

  // Валидация на входните данни
  const errors = validateCarInput(req.body, false);
  if (errors.length > 0)
    return res.status(400).json({
      success: false,
      error: { message: 'Грешки в данните.', code: 'VALIDATION_ERROR',
        details: errors }
    });

  const [result] = await pool.query(
    `INSERT INTO cars (brand, model, year, license_plate, price_per_day,
      type, seats, transmission, fuel_type, mileage,
      image_url, description, available)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
    [brand, model, year, license_plate||null, price_per_day,
      type||'sedan', seats||5, transmission||'automatic',
      fuel_type||'petrol',
      mileage||0, image_url||null, description||null, available!==false]
  );
  res.status(201).json({ success: true, message: 'Автомобилът е добавен.',
    id: result.insertId });
}));
```



UPDATE — редактиране на съществуващ автомобил:

```
// PUT /api/cars/:id
router.put('/:id', requireAuth, requireAdmin, asyncHandler(async (req, res)
=> {
  const carId = parseInt(req.params.id);
  if (isNaN(carId)) return res.status(400).json({ /* невалидно ID */ });

  const errors = validateCarInput(req.body, true); // isUpdate=true
  if (errors.length > 0) return res.status(400).json({ /* грешки */ });

  const [result] = await pool.query(
    `UPDATE cars SET brand=?, model=?, year=?, license_plate=?,
      price_per_day=?, type=?, seats=?, transmission=?,
      fuel_type=?, mileage=?, image_url=?, description=?, available=?
    WHERE id=?`,
    [brand, model, year, license_plate, price_per_day, type, seats,
      transmission, fuel_type, mileage, image_url, description, available,
      carId]
  );
  if (result.affectedRows === 0)
    return res.status(404).json({ success: false,
      error: { message: 'Автомобилът не е намерен.', code: 'NOT_FOUND' }
    });
  res.json({ success: true, message: 'Автомобилът е обновен.' });
}));
```

DELETE — изтриване на автомобил:

```
// DELETE /api/cars/:id
router.delete('/:id', requireAuth, requireAdmin, asyncHandler(async (req,
res) => {
  const carId = parseInt(req.params.id);
  const [result] = await pool.query('DELETE FROM cars WHERE id = ?',
[carId]);
  if (result.affectedRows === 0)
    return res.status(404).json({ success: false,
      error: { message: 'Автомобилът не е намерен.', code: 'NOT_FOUND' }
    });
  res.json({ success: true, message: 'Автомобилът е изтрит.' });
}));
```

Редакцията на съществуващ автомобил се реализира чрез модален прозорец, съдържащ цялостна форма с полета за всички атрибути от таблицата cars – марка, модел, година,



регистрационен номер, цена на ден, тип купе, брой места, скоростна кутия, вид гориво, пробег, URL на изображение, описание и статус на наличност, позволявайки модификация без напускане на текущата страница. *Виж снимка номер 5.4.7.*

CRUD за заявки за наем (routes/requests.js)

Логиката за заявките е най-сложната в проекта — тя включва валидация на данни, проверка на наличност и изчисляване на цена с отстъпка:

CREATE — създаване на нова заявка:

```
// POST /api/requests
router.post('/', requireAuth, asyncHandler(async (req, res) => {
  const { template_id, car_ids, start_date, end_date, notes } = req.body;

  // 1. Валидация на датите и избраните автомобили
  const errors = validateRequestInput(req.body);
  if (errors.length > 0) return res.status(400).json({ /* грешки */ });

  // 2. Проверка дали ВСИЧКИ избрани автомобили са налични
  const [cars] = await pool.query(
    'SELECT * FROM cars WHERE id IN (?) AND available = TRUE',
    [car_ids]
  );
  if (cars.length !== car_ids.length)
    return res.status(400).json({ success: false,
      error: { message: 'Някои автомобили не са налични.' }
    });

  // 3. Изчисляване на цената
  const days = Math.ceil(
    (new Date(end_date) - new Date(start_date)) / (1000 * 60 * 60 * 24)
  );
  const basePrice = cars.reduce((s, c) => s + parseFloat(c.price_per_day),
    0);

  // 4. Прилагане на шаблонна отстъпка
  let discount = 0;
  if (template_id) {
    const [tmpl] = await pool.query('SELECT * FROM templates WHERE
id=?', [template_id]);
    if (tmpl.length > 0) discount = tmpl[0].discount_percent || 0;
  }
});
```



```
const totalPrice = basePrice * days * (1 - discount / 100);

// 5. Запис в rental_requests
const [result] = await pool.query(
  'INSERT INTO rental_requests
(user_id,template_id,start_date,end_date,',
  + 'total_price,notes) VALUES (?, ?, ?, ?, ?, ?)',
  [req.user.id, template_id||null, start_date, end_date,
  totalPrice.toFixed(2), notes||null]
);

// 6. Запис в request_cars (много-към-много)
for (const car_id of car_ids) {
  await pool.query(
    'INSERT INTO request_cars (request_id, car_id) VALUES (?, ?)',
    [result.insertId, car_id]
  );
}

res.status(201).json({
  success: true, message: 'Заявката е създадена.',
  id: result.insertId, total_price: totalPrice.toFixed(2)
});
});
```

READ — четене на заявките. Клиентът вижда само своите, администраторът вижда всички:

```
// GET /api/requests/my (клиент)
router.get('/my', requireAuth, asyncHandler(async (req, res) => {
  const [requests] = await pool.query(
    `SELECT rr.*, t.name AS template_name
FROM rental_requests rr
LEFT JOIN templates t ON rr.template_id = t.id
WHERE rr.user_id = ?
ORDER BY rr.created_at DESC`,
    [req.user.id]
  );
  // Добавяне на автомобилите за всяка заявка
  for (const req of requests) {
    const [cars] = await pool.query(
      `SELECT c.* FROM cars c
```



```
        JOIN request_cars rc ON c.id = rc.car_id
        WHERE rc.request_id = ?`, [req.id]
    );
    req.cars = cars;
  }
  res.json({ success: true, data: requests, count: requests.length });
});

// GET /api/requests (само админ, с филтър по статус)
router.get('/', requireAuth, requireAdmin, asyncHandler(async (req, res) => {
  const { status } = req.query;
  let query = `SELECT rr.*, u.first_name, u.last_name, u.email
                FROM rental_requests rr
                LEFT JOIN users u ON rr.user_id = u.id`;
  const params = [];
  if (status) { query += ' WHERE rr.status = ?'; params.push(status); }
  query += ' ORDER BY rr.created_at DESC';
  const [requests] = await pool.query(query, params);
  /* ...добавяне на автомобилите... */
  res.json({ success: true, data: requests });
}));
```

UPDATE (PATCH) — промяна на статус от администратора:

```
// PATCH /api/requests/:id/status
router.patch('/:id/status', requireAuth, requireAdmin,
  asyncHandler(async (req, res) => {
    const { status } = req.body;
    const VALID =
    ['pending', 'approved', 'rejected', 'completed', 'cancelled'];
    if (!VALID.includes(status))
      return res.status(400).json({ success: false,
        error: { message: 'Невалиден статус.', code: 'INVALID_VALUE' }
      });

    const [result] = await pool.query(
      'UPDATE rental_requests SET status=? WHERE id=?',
      [status, req.params.id]
    );
    if (result.affectedRows === 0)
      return res.status(404).json({ /* NOT_FOUND */ });
    res.json({ success: true, message: 'Статусът е обновен.' });
  }));
```



Отмяна от клиента (само за pending заявки):

```
// POST /api/requests/:id/cancel
router.post('/:id/cancel', requireAuth, asyncHandler(async (req, res) => {
  const [reqs] = await pool.query(
    'SELECT * FROM rental_requests WHERE id=? AND user_id=?',
    [req.params.id, req.user.id]
  );
  if (reqs.length === 0) return res.status(404).json({ /* NOT_FOUND */ });
  if (reqs[0].status !== 'pending')
    return res.status(400).json({ success: false,
      error: { message: 'Може да отмените само чакащи заявки.' }
    });
  await pool.query(
    'UPDATE rental_requests SET status=? WHERE id=?',
    ['cancelled', req.params.id]
  );
  res.json({ success: true, message: 'Заявката е отменена.' });
}));
```

CRUD за коментари (routes/comments.js)

Коментарите следват прост workflow: клиентът добавя коментар (статус pending), администраторът го одобрява (approved) или отхвърля (rejected/delete). На клиентите се показват само approved коментари:

```
// GET /api/comments/platform - публичен, само одобрени
router.get('/platform', asyncHandler(async (req, res) => {
  const [comments] = await pool.query(
    `SELECT pc.*, u.first_name, u.last_name
    FROM platform_comments pc
    LEFT JOIN users u ON pc.user_id = u.id
    WHERE pc.status = 'approved'
    ORDER BY pc.created_at DESC`
  );
  res.json({ success: true, data: comments });
}));

// POST /api/comments/platform - регистриран потребител
router.post('/platform', requireAuth, asyncHandler(async (req, res) => {
  const { content, rating } = req.body;
  const errors = validateCommentInput(req.body);
```



```
if (errors.length > 0) return res.status(400).json({ /* грешки */ });

await pool.query(
  'INSERT INTO platform_comments (user_id, content, rating) VALUES'
  '(?,?,?)',
  [req.user.id, content, rating || null]
);
res.status(201).json({ success:true,
  message: 'Коментарът ще бъде публикуван след одобрение.' });
});

// PATCH /api/comments/:id/approve - само админ
router.patch('/:id/approve', requireAuth, requireAdmin,
  asyncHandler(async (req, res) => {
    const table = req.body.type === 'template'
      ? 'template_comments' : 'platform_comments';
    await pool.query(`UPDATE ${table} SET status='approved' WHERE id=?`,
      [req.params.id]);
    res.json({ success: true, message: 'Коментарът е одобрен.' });
  })
);
```

Клиентски API модул (frontend/js/main.js)

От страна на клиента, всички HTTP заявки минават през централизиран обект `api`, дефиниран в `main.js`. Той автоматично добавя `Authorization` хедъра и обработва стандартизирания формат на грешките:

```
// frontend/js/main.js - извадка от обекта api
const api = {
  async request(endpoint, options = {}) {
    const headers = { 'Content-Type': 'application/json',
    ...options.headers };
    if (auth.token) headers['Authorization'] = `Bearer ${auth.token}`;

    const response = await fetch(`${API_BASE}${endpoint}`, {
      ...options, headers, credentials: 'include'
    });
    const data = await response.json();

    if (!response.ok) {
```



```
const info = data.error || {};  
// Автоматично излизане при изтекъл токен  
if (info.code === 'TOKEN_EXPIRED' || info.code === 'TOKEN_INVALID')  
  auth.logout();  
if (info.details?.length)  
  throw new Error(`${info.message}:  
${info.details.map(e=>e.message).join(', ')}`);  
throw new Error(info.message || 'Грешка');  
}  
// Разопаковане на унифицирания wrapper {success, data}  
return (data.success === true && data.data !== undefined) ? data.data :  
data;  
},  
get: (ep) => this.request(ep),  
post: (ep, body) => this.request(ep, { method: 'POST', body:  
JSON.stringify(body) }),  
put: (ep, body) => this.request(ep, { method: 'PUT', body:  
JSON.stringify(body) }),  
patch: (ep, body) => this.request(ep, { method: 'PATCH', body:  
JSON.stringify(body) }),  
delete: (ep) => this.request(ep, { method: 'DELETE' }),  
};
```

Пример за извикване от `create-request.html` — изчисляване на цена в реално време при промяна на избраните автомобили и дати:

```
// frontend/create-request.html - функция за изчисляване на цена  
function updatePrice() {  
  const startDate = document.getElementById('start_date').value;  
  const endDate = document.getElementById('end_date').value;  
  if (!startDate || !endDate || selectedCars.length === 0) return;  
  
  const days = Math.ceil(  
    (new Date(endDate) - new Date(startDate)) / (1000 * 60 * 60 * 24)  
  );  
  if (days <= 0) return;  
  
  let basePrice = 0;  
  selectedCars.forEach(id => {  
    const car = allCars.find(c => c.id.toString() === id);  
    if (car) basePrice += car.price_per_day;  
  });  
};
```



```
const discount = selectedTemplate?.discount_percent || 0;
const total = basePrice * days * (1 - discount / 100);

document.getElementById('totalPrice').textContent =
    `€${total.toFixed(2)} (${days} дни × ${selectedCars.length}
автомобил/а)`;
}
```

Таблица 2.2.5.1. Пълен списък на REST API endpoints

Метод	Endpoint	Описание	Достъп
POST	/api/auth/register	Регистрация на клиент	Публичен
POST	/api/auth/login	Вход, издаване на JWT	Публичен
GET	/api/auth/me	Текущ потребител	Авт.
GET	/api/cars	Всички автомобили (с филтри)	Публичен
GET	/api/cars/:id	Детайли за един автомобил	Публичен
POST	/api/cars	Добавяне на автомобил	Админ
PUT	/api/cars/:id	Редактиране на автомобил	Админ
DELETE	/api/cars/:id	Изтриване на автомобил	Админ
GET	/api/templates	Всички активни шаблони	Публичен
GET	/api/templates/:id	Детайли за шаблон	Публичен
POST	/api/templates	Добавяне на шаблон	Админ
PUT	/api/templates/:id	Редактиране на шаблон	Админ
DELETE	/api/templates/:id	Изтриване на шаблон	Админ



POST	/api/requests	Създаване на заявка	Клиент
GET	/api/requests/my	Моите заявки	Клиент
GET	/api/requests	Всички заявки	Админ
PATCH	/api/requests/:id/status	Смяна на статус	Админ
POST	/api/requests/:id/cancel	Отмяна на заявка	Клиент
GET	/api/comments/platform	Одобрени коментари	Публичен
POST	/api/comments/platform	Добавяне на коментар	Авт.
GET	/api/comments/pending	Чакащи коментари	Админ
PATCH	/api/comments/:id/approve	Одобряване на коментар	Админ
DELETE	/api/comments/:id	Изтриване на коментар	Админ
POST	/api/contacts	Изпращане на съобщение	Публичен
GET	/api/contacts	Всички съобщения	Админ
PATCH	/api/contacts/:id/read	Маркиране като прочетено	Админ
GET	/api/costs/my	Моите разходи	Клиент
GET	/api/costs/clients	Разходи по клиенти	Админ
GET	/api/admin/dashboard	Dashboard статистики	Админ

2.2.6. Графичен дизайн и структура на приложението

Потребителският интерфейс на "Luxe Auto Resort" е реализиран с HTML5, CSS3 и JavaScript, следвайки принципите на Mobile-First дизайна. Визуалната идентичност на приложението е изградена около тъмна цветова палитра, създаваща усещане за луксозност и изисканост.

Цветовата схема е дефинирана чрез CSS Custom Properties (CSS Variables) в корена на документа. Основният фон е тъмно сив (#171614), акцентният цвят е стоманено син (#4B88A2), цветът на картите е #252525, а СТА (Call-to-Action) цветът е топло оранжев



(#E98A15). Тази палитра осигурява силен контраст за четимост при запазване на луксозната визия.

CSS Variables дефиниция от style.css:

```
:root {
  --bg:          #171614; /* Основен фон */
  --accent:      #4B88A2; /* Акцентен цвят (синьо) */
  --card:        #252525; /* Фон на карти */
  --cta:         #E98A15; /* Call-to-Action (оранжево) */
  --text-light:  #ECE5F0; /* Основен текст */
  --text-dark:   #171614; /* Текст върху светъл фон */
  --success:     #28a745;
  --danger:      #dc3545;
  --transition: all 0.3s ease;
}
```

Навигацията е реализирана чрез фиксиран хедър (position: sticky) с логото и навигационното меню. На мобилни устройства хедърът включва hamburger бутон, показващ/скриващ навигацията. Хедърът динамично добавя елементи въз основа на автентикационното състояние.

Каталогът с автомобили използва CSS Grid за адаптивно разполагане на картите. Всяка карта съдържа снимка, марка и модел, ключови характеристики, цена на ден и бутон за детайли. Responsive дизайнът е реализиран чрез Media Queries с три основни breakpoints: 1024px (tablet), 768px (mobile landscape) и 480px (mobile portrait). На мобилни устройства всички бутони достигат минималната препоръчана ширина за докосване от 48px.

3. Заключение

3.1. Постигнати цели

В хода на разработката на уеб приложението "Luxe Auto Resort" са успешно реализирани всички поставени в началото цели. Създаден е пълнофункционален уеб продукт, демонстриращ умения за пълноциклова разработка на модерно уеб приложение с реална бизнес стойност.

От техническа гледна точка, успешно са постигнати следните резултати: изграден е стабилен REST API сървър с Node.js и Express, реализиращ всички необходими endpoints;



проектирана и реализирана е нормализирана MySQL база данни с девет таблици, правилно дефинирани индекси и референтна цялост чрез чужди ключове; разработен е responsive потребителски интерфейс, работещ на всички устройства; имплементирана е сигурна система за автентикация и авторизация с JWT и bcrypt; и реализиран е пълен набор CRUD операции с централизирана обработка на грешки и стандартизирани API отговори.

Проектът успешно демонстрира интеграцията на съвременен технологичен стек: Node.js, Express, MySQL, HTML5, CSS3 и JavaScript. Кодът следва добрите практики: разделение на concerns, DRY принцип, централизирана обработка на грешки, валидация на входните данни и защита срещу основните уеб уязвимости.

3.2. Бъдещо развитие

"Luxe Auto Resort" предоставя стабилна и разширяема основа за бъдещо развитие. Приоритетна функционалност за следващата версия е интегрирането на онлайн платежна система (Stripe или PayPal). Втора важна насока е системата за email известия чрез Nodemailer. Трета перспектива е разработката на мобилно приложение (React Native или Flutter), използващо същия REST API сървър. Четвърта насока е разширената аналитика с интерактивни графики за визуализация на приходите. Пета перспектива е реализацията на резервационен календар за предотвратяване на двойно резервиране.



4. Използвана литература

4.1. Node.js

- Node.js Official Documentation – <https://nodejs.org/en/docs/>

4.2. Express.js

- Express.js Official Documentation – <https://expressjs.com/en/guide/>
- Express Routing Guide – <https://expressjs.com/en/guide/routing.html>

4.3. MySQL

- MySQL Reference Manual – <https://dev.mysql.com/doc/>
- MySQL2 npm package – <https://www.npmjs.com/package/mysql2>

4.4. HTML5 и CSS3

- MDN Web Docs – HTML – <https://developer.mozilla.org/en-US/docs/Web/HTML>
- MDN Web Docs – CSS – <https://developer.mozilla.org/en-US/docs/Web/CSS>
- CSS-Tricks – A Complete Guide to Flexbox – <https://css-tricks.com/>

4.5. JavaScript

- MDN Web Docs – JavaScript – <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- JavaScript.info – <https://javascript.info/>

4.6. JSON Web Tokens (JWT)

- JWT Introduction – <https://jwt.io/introduction>
- jsonwebtoken npm package – <https://www.npmjs.com/package/jsonwebtoken>

4.7. Сигурност – bcrypt

- bcrypt npm package – <https://www.npmjs.com/package/bcrypt>
- OWASP – Password Storage Cheat Sheet – <https://cheatsheetseries.owasp.org/>

4.8. REST API дизайн

- RESTful API Design – <https://restfulapi.net/>
- HTTP Status Codes – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>



4.9. Responsive Design

- Responsive Web Design Basics – <https://web.dev/responsive-web-design-basics/>

4.10. SDLC

- Software Development Life Cycle – <https://www.guru99.com/software-development-life-cycle-tutorial.html>

4.11. Общи ресурси

- Stack Overflow – <https://stackoverflow.com/>
- GitHub Repository – https://github.com/pa6ov/luxe_auto_resort
- npm Registry – <https://www.npmjs.com/>



5. Приложения

Приложение 5.1. Структура на проекта

Проектът е организиран в следната директорна структура:

```
lar/
├─ backend/           (Сървърна логика)
│   ├─ config/       (Конфигурация на базата)
│   ├─ middleware/   (JWT автентикация)
│   ├─ routes/       (API маршрути)
│   ├─ utils/        (Помощни функции)
│   ├─ scripts/      (Помощни скриптове)
│   └─ server.js     (Главен сървърен файл)
├─ database/         (SQL файлове)
│   ├─ schema.sql    (Структура на базата)
│   └─ seed.sql      (Тестови данни)
├─ frontend/        (Клиентска част)
│   ├─ css/style.css (Глобални стилове)
│   ├─ js/main.js    (Главен JS модул)
│   ├─ index.html, cars.html, car-details.html
│   ├─ templates.html, create-request.html
│   ├─ my-requests.html, admin.html
│   └─ login.html, register.html, contacts.html
└─ README.md
```

Приложение 5.2. Тестови сценарии

#	Тест	Очакван резултат	Статус
1	Регистрация с валидни данни	Успешна регистрация, пренасочване	✓
2	Регистрация с дублиран имейл	Грешка: имейлът е зает	✓
3	Вход с валидни данни	JWT токен, пренасочване	✓



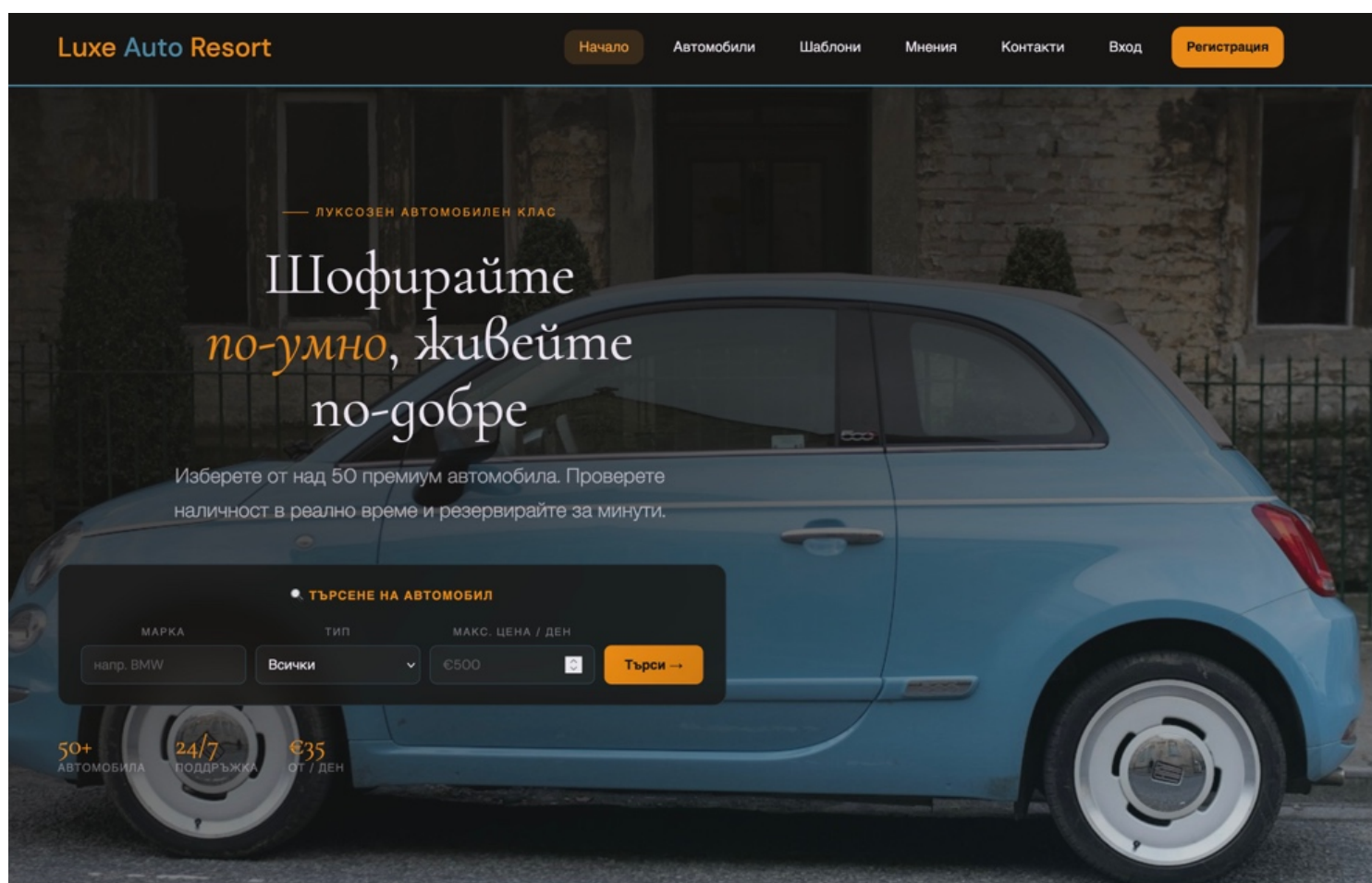
4	Вход с грешна парола	Грешка: невалидни данни	✓
5	Достъп до /admin без роля	Пренасочване към вход	✓
6	Преглед на каталога	Показват се всички автомобили	✓
7	Филтриране по тип	Само избраният тип	✓
8	Филтриране по цена	Само в диапазона	✓
9	Създаване на заявка	Заявка с изчислена цена	✓
10	Добавяне на автомобил (админ)	Автомобилът се появява	✓
11	Изтриване на автомобил (админ)	Автомобилът изчезва	✓
12	Одобряване на заявка (админ)	Статусът се обновява	✓
13	Отмяна на заявка (клиент)	Статусът е 'отменена'	✓
14	Оставяне на коментар	Коментарът чака одобрение	✓
15	Одобряване на коментар	Коментарът е видим	✓
16	Изпращане на контактно съобщение	Съобщението е записано	✓
17	Изчисляване с шаблонна отстъпка	Цената включва отстъпката	✓
18	Responsive дизайн (мобилен)	Интерфейсът се адаптира	✓



Приложение 5.3. Тестови акаунти

Роля	Имейл	Парола
Администратор	admin@luxecauto.bg	admin123
Клиент 1	ivan@abv.bg	ivan123
Клиент 2	maria@abv.bg	mariya123

Приложение 5.4. Визуализация на потребителския интерфейс



Фиг. 5.4.1. Начална страница с интегрирана търсачка и навигация



Luxe Auto Resort Начало **Автомобили** Шаблони Мнения Контакты Вход **Регистрация**

ФИЛТРИ

Марка Тип Мин. цена / ден Макс. цена / ден Сортиране

напр. Mercedes Всички модели 0 500 По погразбиране Приложи

48 автомобили

Renault Clio

sedan 2015 5 места automatic diesel

€45.00 / ден

Виж детайли

Opel Corsa

sedan 2014 5 места automatic petrol

€40.00 / ден

Виж детайли

Fiat Bravo

sedan 2011 5 места automatic petrol

€35.00 / ден

Виж детайли

Фиг. 5.4.2. Каталог с автомобили и филтриращи механизми

ID	Автомобили	Период	Пакет	Цена	Статус	Дата	Действия
#1	Renault Clio	10 юли 2025 г. — 13 юли 2025 г. 3 дни	Уикенд	€121.50	ОДОБРЕНА	6 април 2026 г.	—
#3	Fiat Bravo	20 юли 2025 г. — 25 юли 2025 г. 5 дни	—	€175.00	ЧАКАЩА	6 април 2026 г.	× Отмени
#5	Toyota Land Cruiser	1 август 2025 г. — 15 август 2025 г. 14 дни	Бизнес	€2310.00	ОДОБРЕНА	6 април 2026 г.	—
#7	VW Passat	1 юни 2025 г. — 4 юни 2025 г. 3 дни	Уикенд	€148.50	ПРИКЛЮЧЕНА	6 април 2026 г.	—
#9	VW Golf	1 юли 2025 г. — 8 юли 2025 г. 7 дни	Седмичен	€357.00	ОТМЕНЕНА	6 април 2026 г.	—

Фиг. 5.4.3. Потребителски профил (редакция на данни)



Профил Статистика Сигурност

ИИ Качи снимка
Препоръчителен размер: 200x200px

Име * Иван
Фамилия * Иванов

Имейл ivan@abv.bg
Телефон +359888222222

Запази промените

Фиг. 5.4.4. Управление на заявки от потребителска гледна точка

Dashboard Автомобили Шаблини Заявки Разходи Коментари Съобщения

Управление на автомобили + Добави автомобил

ID	Автомобил	Цена/ден	Тип	Статус	Недостъпен до	Действия
1	Renault Clio	€45.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
2	Opel Corsa	€40.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
3	Fiat Bravo	€35.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
4	VW Golf	€38.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
5	VW Golf	€38.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
6	Citroen C4	€35.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
7	Citroen C4	€42.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
8	VW Passat	€55.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
9	VW Golf	€55.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
10	VW Golf	€58.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий
11	Toyota Auris	€70.00	sedan	НАЛИЧЕН	—	Редактирай Изтрий

Фиг. 5.4.5. Административно управление на автопарка



Dashboard	Автомобили	Шаблони	Заявки	Разходи	Коментари	Съобщения
Всички заявки						
Всички статуси ▾						
ID	Клиент	Автомобили	Период	Цена	Статус	Действия
1	Иван Иванов	Renault Clio	10 юли 2025 г. – 13 юли 2025 г.	€121.50	ОДОБРЕНА	Приключи
2	Мария Петрова	Opel Corsa	15 юли 2025 г. – 22 юли 2025 г.	€238.00	ОДОБРЕНА	Приключи
3	Иван Иванов	Fiat Bravo	20 юли 2025 г. – 25 юли 2025 г.	€175.00	ЧАКАЩА	Одобри Откажи
4	Мария Петрова	BMW 320	5 юли 2025 г. – 12 юли 2025 г.	€446.25	ОДОБРЕНА	Приключи
5	Иван Иванов	Toyota Land Cruiser	1 август 2025 г. – 15 август 2025 г.	€2310.00	ОДОБРЕНА	Приключи
6	Мария Петрова	BMW X3	28 юли 2025 г. – 3 август 2025 г.	€780.00	ЧАКАЩА	Одобри Откажи
7	Иван Иванов	VW Passat	1 юни 2025 г. – 4 юни 2025 г.	€148.50	ПРИКЛЮЧЕНА	
8	Мария Петрова	BMW X5	10 юли 2025 г. – 17 юли 2025 г.	€1260.00	ОТКАЗАНА	
9	Иван Иванов	VW Golf	1 юли 2025 г. – 8 юли 2025 г.	€357.00	ОТМЕНЕНА	

Фиг. 5.4.6. Административно управление на всички заявки



Редактирай автомобил

Марка *

Renault

Модел *

Clio

Година *

2015

Цена/ден *

45.00

Тип

Сеган

Места

5

Скоростна кутия

Автоматик

Гориво

Дизел

Снимка (URL или локален път)

images/renault-clio-2015-1.webp

Описание

Икономичен хечбек с автоматик. Перфектен за града с нисък разход на гориво. Включва климатик, ел. стъкла и централно заключване.

СТАТУС НА АВТОМОБИЛА

Текущ статус

Наличен

Запази

Фиг. 5.4.7. Модален прозорец за редакция на автомобил